

Route Handler とは

なぜ API を経由するのか

ここまでは `page.tsx` の中で `MOCK_SHOPS` を直接 `import` していました。これは「データの取り出し方」が画面のコードに埋め込まれている状態です。

データの取得方法	変更が必要な範囲
ページから直接 <code>import</code>	データ源を変えると、使っている全ページを書き換える
Route Handler 経由	フロントエンドは <code>/api/shops</code> を呼ぶだけ。書き換えるのは API 側のみ

フロントエンドとデータ源の間に `API` を1枚はさむことで、両者を切り離せます。モックデータを後でデータベースに差し替えても、ページ側のコードは1行も変わりません。

実務では、`API` が別のサーバーやチームの担当になることもあります。「`URL` を叩けばデータが返る」という形にしておけば、中身が何であってもフロントエンドは同じように書けます。

route.ts

page.tsx と route.ts

`Next.js` では `app/` 配下に `page.tsx` と `route.ts` を配置できますが、機能や役割がまったく違います。

項目	page.tsx	route.ts
返すもの	画面 (HTML)	データ (JSON など)
主な利用者	ブラウザで URL にアクセスした人	fetch などのプログラム
実行される場所	サーバー (Client Component はブラウザでも)	サーバーのみ
画面の見た目	ある	ない

route.ts の配置

`route.ts` を `app/` 以降のフォルダに配置すると、そのフォルダ階層がそのままAPIのURL になります。

ファイル	エンドポイント
<code>app/api/users/route.ts</code>	<code>/api/users</code>
<code>app/api/shops/[id]/route.ts</code>	<code>/api/shops/1</code>

API のエンドポイントは慣習として `app/api/` 配下にまとめますが、`api` である必要はありません。

サーバー上だけで実行される

`route.ts` はバックエンドの機能のため、コードがブラウザに送られることはありません。データベースの接続情報や外部サービスの `API` キーなど、外に出したくない情報を扱うのに向いています。

page.tsx との共存ルール

同じ階層に `page.tsx` と `route.ts` を同時には置けません。どちらも同じ `URL` を担当しようとして競合するためです。

状況	可否
<code>app/shop/page.tsx</code> のみ	○（画面）
<code>app/api/shops/route.ts</code> のみ	○（API）
<code>app/shop/page.tsx</code> と <code>app/shop/route.ts</code> を同じ階層に	✕（ビルドエラー）

HTTPメソッドと動的セグメント

対応する HTTP メソッド

`route.ts` の中では、`HTTP` メソッドと同じ名前の関数を `export` します。どのメソッドに対応させるかは、関数名で決まります。

関数名	意味	主な用途
GET	取得	一覧・詳細の取得
POST	作成	新規データの登録
PUT	更新（全体差し替え）	既存データの更新
PATCH	更新（一部差し替え）	既存データの一部だけ更新
DELETE	削除	データの削除

1つの `route.ts` の中に、複数のメソッドの関数を一緒に書けます。

Request と Response

`Route Handler` の関数は、`Request` を受け取って `Response` を返すだけの仕組みです。Web 標準の書き方がそのまま使えます。

```
export async function GET(request: Request) {  
  // /api/shops?area=osaka の "osaka" を取り出す  
  const { searchParams } = new URL(request.url)  
  const area = searchParams.get("area")  
  
  return NextResponse.json({ area })  
}
```

項目	意味
request	リクエストの情報（URL・ヘッダー・ボディなど）を持つオブジェクト
searchParams	クエリパラメータ（?の後ろ）を取り出すためのオブジェクト
NextResponse	標準の Response を拡張したもの。json() などの便利なメソッドが追加されている

`request` を使わない場合は、引数ごと省略できます。

動的セグメント

`Route Handler` 側でも `params` を `Promise` として受け取り、`[id]` フォルダでのルーティングを利用できます。

GET /api/shops/1 など

```
// app/api/shops/[id]/route.ts
export async function GET(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params
  const shop = MOCK_SHOPS.find((shop) => shop.id === Number(id))

  if (!shop) {
    return NextResponse.json({ error: "Not Found" }, { status: 404 })
  }

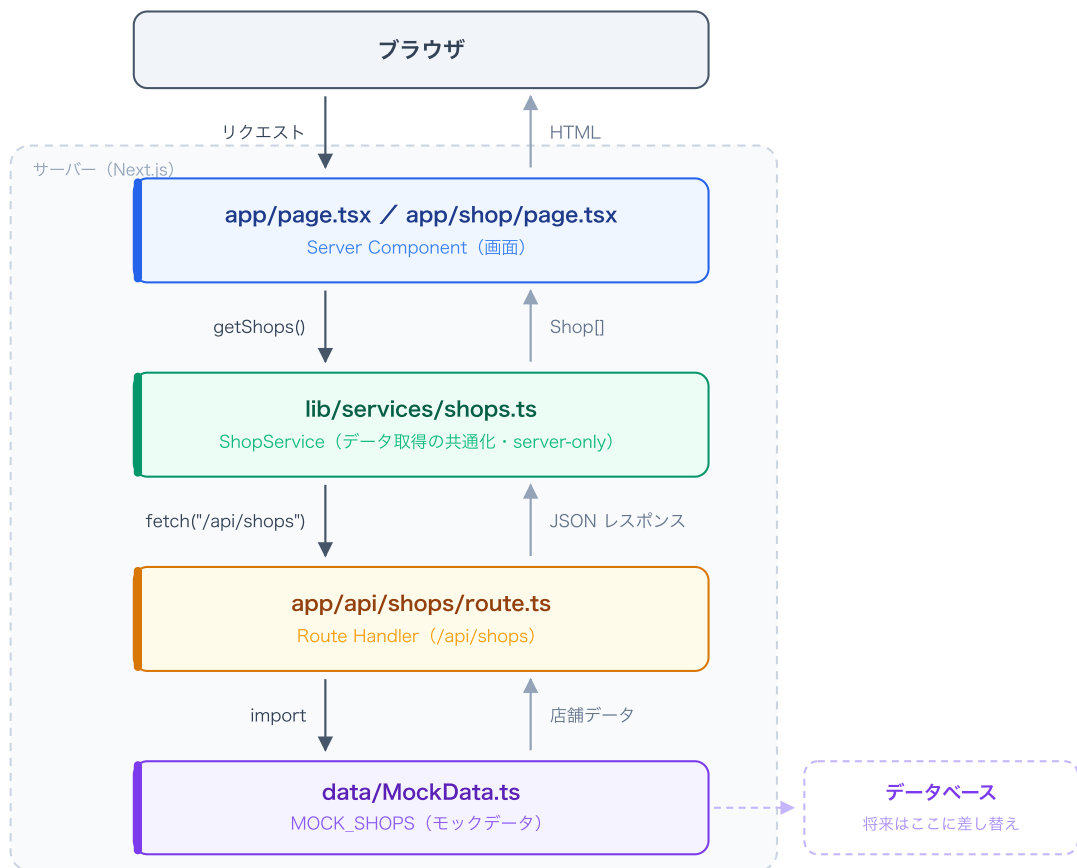
  return NextResponse.json({ shop })
}
```

項目	意味
第2引数	動的セグメントの値を params として受け取る
status	レスポンスの HTTP ステータスコードを指定する（省略時は 200）

データが見つからなかった場合に **404** を返すなど、**API** は「正常系以外」も返せるようにしておきます。

APIサーバ作成

フロー



GET ハンドラの作成

`app/api/shops/route.ts` を新規作成します。

```
✓ app
  > (customer)
  > admin
  ✓ api
    ✓ shops
      TS route.ts
```

```
import { NextResponse } from "next/server"
// モックデータインポート
import { MOCK_SHOPS } from "@/data/MockData"

export async function GET() {
  const shops = MOCK_SHOPS
  // JSON でレスポンス
  return NextResponse.json({ shops })
}
```

項目	意味
GET 関数	<code>GET /api/shops</code> へのリクエストを処理する
<code>NextResponse.json()</code>	オブジェクトを JSON レスポンスに変換して返す

`/api/shops` に直接ブラウザでアクセスすると、`{ "shops": [...] }` という JSON が返ってくることを確認できます。

環境変数

Server Component と URL

Server Component から fetch で API にリクエストする場合は、`http://localhost:3000/api/shops` のような完全なURLを指定します。

```
const response = await fetch(
  `${process.env.NEXT_PUBLIC_APP_URL}/api/shops`
);
```

Server Component はサーバー上で実行されるため、現在のURLを基準とした相対URLを使用できません。

NEXT_PUBLIC の定義

Next.js では、ブラウザ側から環境変数を利用する場合、環境変数名に `NEXT_PUBLIC_` を付けます。

`.env.local` に `NEXT_PUBLIC_APP_URL` を追加します。

```
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

サーバー側だけで使用する環境変数には、`NEXT_PUBLIC_` をつける必要はありません。

NEXT_PUBLIC の取得

プログラムからは、次のように値を取得できます。

```
const appUrl = process.env.NEXT_PUBLIC_APP_URL;
```

ShopService の作成

`/`（トップページ）と `/shop`（店舗一覧）の両方で、同じ「店舗一覧を取得する」処理が必要になります。ページごとに `fetch` を書くと **同じコードが重複するため**、`lib/services/shops.ts` に共通化しておきます。

```
import "server-only"
import type { Shop } from "@types/shop"

// ベースURL を設定
const BASE_URL = process.env.NEXT_PUBLIC_APP_URL ?? "http://localhost:3000"

// 店舗一覧取得処理の共通化
export async function getShops(): Promise<Shop[]> {
  // /api/shop から店舗データ取得（JSON）
  const res = await fetch(`${BASE_URL}/api/shops`, { cache: "no-store" })
  const data: { shops: Shop[] } = await res.json()
  return data.shops
}
```

server-only

`server-only` を付けておくことでサーバー専用コードを **Client Component** から利用させないようにできます。

```
import "server-only"
```

項目	意味
<code>import "server-only"</code>	このファイルをブラウザ側のコードから誤って読み込んだ場合にビルドエラーにする
<code>cache: "no-store"</code>	毎回最新のデータを取得する（キャッシュしない）

戻り値の型

`async` を付けた関数は、必ず `Promise` を返します。中で `return data.shops` と書いていても、実際に返るのは `Shop[]` そのものではなく、それを包んだ `Promise` です。

```
export async function getShops(): Promise<Shop[]> {
```

`Promise<Shop[]>` の `< >` の中には、待った結果として得られる値の型を書きます。

書き方	型	意味
<code>getShops()</code>	<code>Promise<Shop[]></code>	まだ結果が出ていない状態
<code>await getShops()</code>	<code>Shop[]</code>	待った後の、店舗の配列

`Promise` は「後で結果を渡す」という引換券のようなものです。`await` を書くと、その券を実際の値に引き換えます。

await の付け忘れ

型を書いておく利点は、ここで効いてきます。

```
// x await を忘れている
const shops = getShops()
shops.map(...) // エラー: Promise には map がない
```

`shops` の型が `Promise<Shop[]>` のままなので、`map` を呼んだ時点でエディタが赤線を出します。型を書いていないと、実行するまで気づけません。

ブラウザで `shops.map is not a function` というエラーが出たときは、`await` の付け忘れをまず疑ってください。

res.json() の型

同じファイルの中に、もう1か所 `Promise` が出てきます。

```
const data: { shops: Shop[] } = await res.json()
```

`res.json()` の戻り値は `Promise<any>` です。`API` から何が返ってくるかを `TypeScript` は知らないため、`any`（何でもあり）になっています。

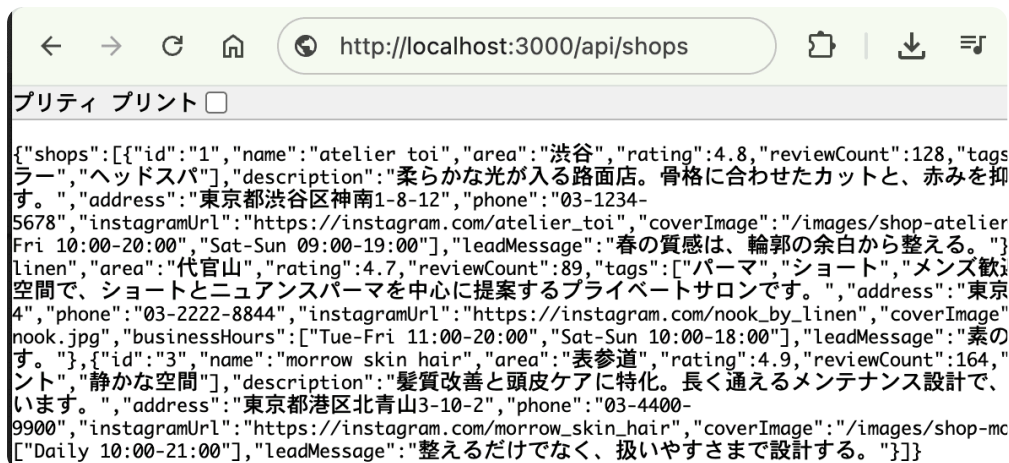
そのままでは補完も型チェックも効かないので、`{ shops: Shop[] }` と受け取る側で型を宣言しています。`Route Handler` が `NextResponse.json({ shops: MOCK_SHOPS })` を返しているの
で、その形に合わせた指定です。

記述	型
<code>res.json()</code>	<code>Promise<any></code>
<code>await res.json()</code>	<code>any</code>
<code>data</code>	<code>{ shops: Shop[] }</code>

`API` の戻り値の型は自動では決まりません。「この `URL` はこのフォーマットの `JSON` を返す」とように開発者が型を決めます

API 動作確認

`http://localhost:3000/api/shops` にアクセスして、APIからデータ取得できるか確認します。



トップページを開いたとき、開発者ツールの **Network** タブに `/api/shops` は表示されません。この **fetch** はサーバーの中で完結しているためです。 **Network** タブに出るのは、ブラウザから直接 `/api/shops` を開いた場合だけです。

トップページ

Featured Shops

今日の気分から選べるサロン

エリアやメニューから、あなたにぴったりのサロンを見つけましょう。

渋谷



atelier toi
春の質感は、輪郭の余白から整える。

似合わせカット 透明感カラー ヘッドスパ

柔らかな光が入る路面店。骨格に合わせたカットと、赤みを抑えたカラー提案が強みです。

評価
4.8 (128件)

詳細を見る

代官山



nook by linen
素の雰囲気、少しだけ意志を足す。

パーマ ショート メンズ歓迎

ミニマルな空間で、ショートとニュアンスパーマを中心に提案するプライベートサロンです。

評価
4.7 (89件)

詳細を見る

表参道



morrow skin hair
整えるだけでなく、扱いやすさまで設計する。

髪質改善 トリートメント 静かな空間

髪質改善と頭皮ケアに特化。長く通えるメンテナンス設計で、忙しい大人世代から支持されています。

評価
4.9 (164件)

詳細を見る

インポートからAPI に切り替え

`app/page.tsx` を、 `MOCK_SHOPS` の直接インポートから `ShopService` 経由に書き換えます。

```

import SectionHeading from "@components/SectionHeading"
import ShopCard from "@components/ShopCard"
// ShopService 読み込み
import { getShops } from "@lib/services/shops"

export default async function Home() {
  // APIから店舗情報取得
  const shops = await getShops()

  return (
    <div>
      <SectionHeading
        eyebrow="Featured Shops"
        title="今日の気分から選べるサロン"
        description="エリアやメニューから、あなたにぴったりのサロンを見つけましょう。"
      />
      <div className="mt-8 grid gap-6 lg:grid-cols-3">
        {shops.map((shop) => (
          <ShopCard key={shop.id} shop={shop} />
        ))}
      </div>
    </div>
  )
}

```

`page.tsx` の関数に `async` を付けて `await getShops()` と書くだけで、`Server Component` の中でデータ取得ができます。`useEffect` も `useState` も不要です。

店舗一覧ページ

Shops

店舗一覧

ターゲット層や店舗のコンセプトに合わせて選べるよう、切り口を変えて提案します。

渋谷



atelier toi
春の質感は、輪郭の余白から整える。

似合わせカット 透明感カラー ヘッドスパ

柔らかな光が入る路面店。骨格に合わせたカットと、赤みを抑えたカラー提案が強みです。

評価
4.8 (128件)

詳細を見る

代官山



nook by linen
素の雰囲気、に、少しだけ意志を足す。

パーマ ショート メンズ歓迎

ミニマルな空間で、ショートとニュアンスパーマを中心に提案するプライベートサロンです。

評価
4.7 (89件)

詳細を見る

表参道



morrow skin hair
整えるだけでなく、扱いやすさまで設計する。

髪質改善 トリートメント 静かな空間

髪質改善と頭皮ケアに特化。長く通えるメンテナンス設計で、忙しい大人世代から支持されています。

評価
4.9 (164件)

詳細を見る

app/shop/page.tsx も同じように書き換えます。

```
import SectionHeading from "@/components/SectionHeading"
import ShopCard from "@/components/ShopCard"
import { getShops } from "@/lib/services/shops"

export default async function ShopPage() {
  const shops = await getShops()

  return (
    <div>
      <SectionHeading
        eyebrow="Shops"
        title="店舗一覧"
        description="ターゲット層や店舗のコンセプトに合わせて選べるよう、切り口を変えて提案します。"
      />
      <div className="mt-8 grid gap-6 lg:grid-cols-3">
        {shops.map((shop) => (
          <ShopCard key={shop.id} shop={shop} />
        ))}
      </div>
    </div>
  )
}
```

まとめ

項目	内容
Route Handler	<code>app/api/</code> 配下の <code>route.ts</code> が API エンドポイントになる仕組み
対応メソッド	GET / POST / PUT / PATCH / DELETE という名前の関数を export する
Request / Response	Route Handler は Web 標準のオブジェクトをそのまま使う
status	<code>NextResponse.json()</code> の第2引数でステータスコードを指定できる
page.tsx との共存	同じ階層に page.tsx と route.ts は同時に置けない
GET	HTTP の GET リクエストを処理する関数名
fetch	標準の HTTP クライアント。axios・ky のような外部ライブラリを使わない
NEXT_PUBLIC_APP_URL	Server Component が自分自身の API を呼ぶときに必要な、実行中のポートに合わせた URL
ShopService	複数ページで共通の「データ取得」処理をまとめた関数群 (<code>lib/services/shops.ts</code>)
Server Component + fetch	状態を持たないページは、 <code>useQuery</code> を使わず <code>async</code> 関数の中で直接 <code>await</code> できる
データ源の差し替え耐性	フロントエンドは <code>Route Handler</code> しか見ていないため、データの中身をDBに差し替えても書き換えが不要